영남대학교 Agentic AI 강의 과정

네이버클라우드 서버에 Claude Code 설치 & 환경 설정 (2주차 - 4일차 이론)

대상: Claude를 처음 쓰는 입문자 학생

일정: 2026. 11. 08(일) 13:00 ~ 18:00

강사: 신인철 (클라우드스퀘어)

> ℹ️ **오늘의 한 줄 목표**
> "내 PC가 아닌 **네이버클라우드 Ubuntu 서버**에서, **AI 에이전트(Claude Code)가 서버를 직접 다루는** 경험을 한다!"

🕐 4일차 — 서버 위의 AI 에이전트, Claude Code

2026. 11. 08(일) 13:00 ~ 18:00 시간표

이론 3시간 · 실습 1.5시간 · 정리 30분 · 50분 수업 + 10분 휴식 · 토 · 일 13:00 ~ 18:00 · 총 8일 / 40시간

시간	구분	내용
13:00 ~ 13:50	이론 1	3일차 회고 · 왜 서버에서 Claude Code인가 · 에이전트 동작 원리
13:50 ~ 14:00	휴식	☕ 10분 휴식
14:00 ~ 14:50	이론 2	접속 구조(Remote SSH · MobaXterm · Mac 터미널) · 설치 · 첫 실행 · 팀 계정 로그인
14:50 ~ 15:00	휴식	☕ 10분 휴식
15:00 ~ 15:50	이론 3	CLAUDE.md 이해 · 권한과 안전 규칙 · 서버 이미지 복구 · GitHub
15:50 ~ 16:00	휴식	☕ 10분 휴식
16:00 ~ 16:50	실습 ①	접속 → Claude Code 설치 → 팀 계정 로그인 (Lab 00~03)
16:50 ~ 17:00	휴식	☕ 10분 휴식
17:00 ~ 17:30	실습 ②	안전 규칙 (Lab 04) → Claude Code로 하루 할 일 메모 앱 v1~v4 (Lab 05)
17:30 ~ 18:00	정리	메모 앱 v3~v4 마무리 (~17:45) → 서버 이미지 생성 · 정리 (Lab 07) · Q&A — v5~v6 · GitHub(Lab 06)는 과제

🗺️ 8일간의 Agentic AI 로드맵 (1~2주차)

네이버클라우드 기초 및 인프라 구축부터 Claude 를 활용한 바이브 코딩까지

1주차 : 네이버클라우드 기초 & 첫 바이브 코딩

Ncloud 인프라 · Nginx 웹서버 · Claude Chat으로 만든 웹사이트 배포

Day 1 10/31(토) 13:00~18:00

네이버클라우드 기초와 웹서버 구축

- 클라우드 컴퓨팅 · Ncloud 개요 · 계정과 콘솔
- 네트워크 기초: IP · VPC · Subnet · ACG · 포트
- 스토리지 기초: Block Storage · Object Storage
- Ubuntu 서버 → Nginx → index.html 공개

핵심 결과물: 웹서버 1대 · 첫 웹페이지 공개

Day 2 11/01(일) 13:00~18:00

Agentic AI와 바이브 코딩으로 웹사이트 개선

- LLM과 Agentic AI 개념 (챗봇 → 에이전트)
- AI 3대장 비교 · 좋은 프롬프트 4요소
- 웹 기초(HTML · CSS · JS) · 배포 원리(HTTP · SSH)
- Claude Chat으로 만든 웹사이트를 서버에 배포

핵심 결과물: 바이브 코딩으로 개선한 웹사이트 공개

2주차 : Claude · Cowork & Claude Code

모델 · 요금제 · Cowork 위임 · 서버 위의 AI 에이전트

Day 3 11/07(토) 13:00~18:00

Claude 기초와 Claude Cowork

- 모델(Opus · Sonnet · Haiku) · 요금제 · 앱 3가지
- Cowork 사용법: 폴더 · 커넥터 · 지침 · 프로젝트
- 승인 화면 읽기 · 안전 수칙
- Cowork로 달력 웹페이지 → 서버 공개

핵심 결과물: 전역 지침 · 프로젝트 · 달력 웹페이지

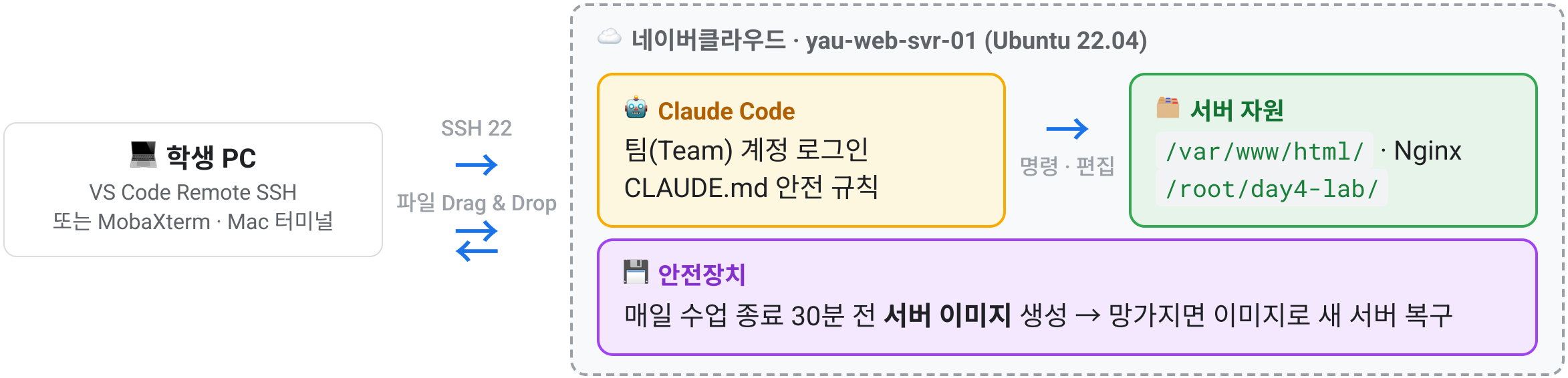
Day 4 11/08(일) 13:00~18:00

서버 위의 AI 에이전트, Claude Code

- 서버에 Claude Code 설치 · 팀 계정 로그인
- CLAUDE.md · 권한 모드 · 안전 규칙
- 메모 앱 v1~v4: Flask API · 서버 저장 · Nginx
- 서버 이미지 복구 · GitHub

핵심 결과물: 달력 연동 메모 앱(서버 저장) · 서버 이미지

🎯 Day 4에서 완성할 결과물



구분	내용
필수	서버 접속 → Claude Code 설치 → 팀 계정 로그인 → 안전 규칙 → **하루 할 일 메모 앱**(달력 연동 · 서버 저장) → 서버 이미지 생성
선택	작업물을 GitHub에 Push (제출 · 포트폴리오)

Day 4 이론 학습 목차

Part	주제	핵심 질문
1	Day 3 회고 · 왜 서버에서 Claude Code인가	내 PC가 아니라 서버에서 쓰는 이유는?
2	Claude 도구 · 터미널 · AI 에이전트 이해	웹 Claude와 Claude Code는 무엇이 다른가?
3	접속 구조: VS Code Remote SSH · MobaXterm · 터미널 `ssh`	서버에 어떻게 들어가고 파일은 어떻게 옮기나?
4	설치 · 첫 실행 · 팀 계정 로그인	브라우저 없는 서버에서 어떻게 로그인하나?
5	**CLAUDE.md 이해하기**	AI에게 규칙을 어떻게 "기억"시키나?
6	권한과 안전 규칙	root 권한의 에이전트를 어떻게 통제하나?
7	서버 이미지로 복구하기	서버가 망가지면 어떻게 되돌리나?
8	GitHub의 역할 & 실습 미리보기	내 작업물은 어디에 남기나?

🧭 Part 1. 왜 서버에서 Claude Code인가

환경 통일 · 학생 PC 보호 · 과정 연속성 · 에이전트 경험

1.1 Day 3 회고 — 내 PC에서 "일을 맡기는" AI를 경험했다

Day 3 이론
Claude 기초

모델

Opus · Sonnet · Haiku
작업에 맞게 고르기

세 가지 방식

Chat(주고받기) ·
Cowork(맡기기) ·
Code(만들기)

Day 3 실습
Cowork 위임

전용 폴더

`Claude-실습` 하나만 연결

좋은 위임

결과물 + 입력 + 뉘앙스

Day 3 안전
승인 · 규칙

승인 화면 읽기

"작업 전 확인" 모드 유지

전역 지침 · 프로젝트

"나만의 규칙"을 저장해
재사용

Day 3 결과물
서버에 공개

calendar.html

Cowork가 만들고
사람이 서버에 업로드

확인

`http://<공인 IP>/calendar.html`

💡 **오늘의 질문**: 어제는 AI가 **내 PC**에서 파일을 만들고 **사람이** 서버에 올렸습니다. 오늘은 AI가 **서버 안에서 직접** 파일을 만들고 고치게 하려면 무엇이 필요할까요? → **Claude Code + CLAUDE.md + 서버 이미지**

1.2 서버에서 사용하는 4가지 이유

이유 1
환경 통일

OS 차이 제거

학생마다 다른 Windows · macOS 대신 모두 **같은 Ubuntu**에서 같은 명령으로 진행

이유 2
학생 PC 보호

연습장 역할

에이전트가 root 권한으로 무엇을 하든 영향은 **서버 안**에서 끝남 — 내 PC는 안전

이유 3
과정 연속성

Day 1~3 서버 재사용

공인 IP · root 비밀번호 · `/var/www/html` 그대로 이어서 사용

이유 4
에이전트 경험

AI가 서버를 직접 다룸

명령 실행 · 파일 편집 · 서비스 점검을 AI에게 맡기고 결과를 검증

💡 **핵심**: 서버는 **"망가져도 되돌릴 수 있는 연습장"** 입니다. 단, 되돌리는 방법(서버 이미지)을 반드시 함께 준비합니다. (Part 7)

1.3 Day 1 → Day 2 → Day 3 → Day 4 연결

Day 1 · 인프라
서버 만들기
VPC · Ubuntu · Nginx
index.html 배포

→

Day 2 · 바이브 코딩
Claude 활용
claude.ai로 코드 생성
프롬프트 · PRD

→

Day 3 · Claude · Cowork
내 PC의 에이전트
모델 · 요금제 이해
Cowork로 작업 위임

→

Day 4 · 오늘
서버 위의 에이전트
Claude Code 설치
AI가 서버를 직접 작업

구분	Day 2 (claude.ai 웹)	Day 3 (Cowork · 내 PC)	Day 4 (Claude Code · 서버)
결과가 생기는 곳	채팅 화면 · 아티팩트	**내 PC 폴더의 실제 파일**	**서버의 실제 파일**
실행	사람이 복사 → 붙여넣기 → 실행	AI가 PC 파일 작업 (승인 후) · 서버 업로드는 사람	**AI가 서버 명령을 직접 실행** (승인 후)
결과 확인	미리보기 화면	내 PC 파일 · `/calendar.html`	`http://<공인 IP>` 실제 웹 · 터미널 결과
사람의 역할	요청 · 복사	범위 지정 · 승인 · 업로드	**요청 · 승인 · 검증**

1.3 Day 1 → Day 2 → Day 3 → Day 4 연결 — 3단 비교

구분	Day 2 (claude.ai 챗)	Day 3 (Cowork · 내 PC)	Day 4 (Claude Code · 서버)
결과가 생기는 곳	채팅 화면 · 아티팩트	**내 PC 폴더의 실제 파일**	**서버의 실제 파일**
실행	사람이 복사 → 붙여넣기 → 실행	AI가 PC 파일 작업 (승인 후) · 서버 업로드는 사람	**AI가 서버 명령을 직접 실행** (승인 후)
결과 확인	미리보기 화면	내 PC 파일 · `/calendar.html`	`http://<공인 IP>` 실제 웹 · 터미널 결과
사람의 역할	요청 · 복사	범위 지정 · 승인 · 업로드	**요청 · 승인 · 검증**

🤖 Part 2. Claude 도구와 AI 에이전트 이해

대화형 AI에서 일하는 AI로

2.1 Claude를 쓰는 방법 비교

방식	어디서	할 수 있는 일	오늘
claude.ai (웹 · 앱)	브라우저 · Claude Desktop	대화, 문서 · 코드 생성, 아티팩트	Day 2
Claude Cowork	Claude Desktop (내 PC)	내 폴더의 파일 · 앱 작업을 맡기기(위임)	Day 3
Claude Code (CLI)	**터미널** (서버 · PC)	폴더의 파일 읽기 · 수정, 명령 실행, 여러 단계 작업	✅
Claude Code (VS Code 확장)	VS Code 패널	CLI와 같은 기능을 편집기 화면에서	선택

- **Claude Code** = 터미널에서 동작하는 **AI 코딩 에이전트**
- 오늘은 서버에 **CLI 버전**을 설치하고, VS Code Remote SSH 터미널 · MobaXterm(Windows) · 터미널 `ssh` (Mac)에서 실행합니다.
- 로그인은 **교육용 팀(Team) 계정 좌석**을 사용합니다.

2.2 AI 에이전트는 어떻게 일하나?

① 사람
요청
"서버 상태를 점검해줘"

→

② Claude
계획
필요한 명령과 파일을 판단

→

③ 도구 실행
승인 후 실행
파일 읽기 · 편집

→

④ 결과 확인
스스로 검증
실행 결과를 보고 다음 행동 결정

→

⑤ 보고
요약 · 다음 제안
사람이 결과를 **검증**

- 챗봇은 **답을 말해주고**, 에이전트는 **일을 직접 하고 결과를 확인**합니다.
- ③ 단계마다 Claude Code는 **"이 명령을 실행해도 될까요?"** 라고 승인을 요청합니다. → 사람이 최종 책임자

2.3 Claude Code가 하는 일 & 사람이 하는 일

Claude Code가 하는 일	사람이 하는 일
폴더 구조 · 파일 내용 읽기	**무엇을 할지** 명확하게 요청
파일 생성 · 수정 (변경 내용 미리 보여줌)	변경 내용 **확인 후 승인 / 거절**
터미널 명령 실행 (`apt`, `systemctl`, `git` ...)	위험한 명령은 **거절** (`rm -rf` 등)
실행 결과를 보고 오류를 스스로 수정	브라우저 · 터미널로 **결과 검증**
작업 내용 요약	안전 규칙(`CLAUDE.md`) 정하기

> ❗ **IMPORTANT**
> AI가 만든 결과는 **반드시 사람이 검증**합니다. "됐다고 했으니 됐겠지"가 아니라 **`http://<공인 IP>`를 직접 열어 확인**하는 습관!

2.4 터미널(CLI)이란? — Claude Code가 사는 곳

- **터미널(Terminal)**: 마우스 대신 **키보드로 명령어를 입력**해 컴퓨터를 움직이는 글자 화면
  - **CLI** (Command-Line Interface, 명령줄 인터페이스)라고도 부릅니다.
- **GUI와의 차이**: GUI는 아이콘을 **클릭**하고, CLI는 명령어를 **입력**합니다.
- 서버에는 보통 **화면(GUI)이 없습니다**. 그래서 서버 관리는 **터미널**에서 합니다.
  - 프로그램 설치 · 파일 만들기 · 서비스 실행 · Git 관리 모두 터미널 작업
- **Claude Code**는 이 터미널 안에서 함께 일하는 **AI 동료**입니다.



2.5 claude.ai(브라우저) vs Claude Code(터미널) — 무엇이 다른가?

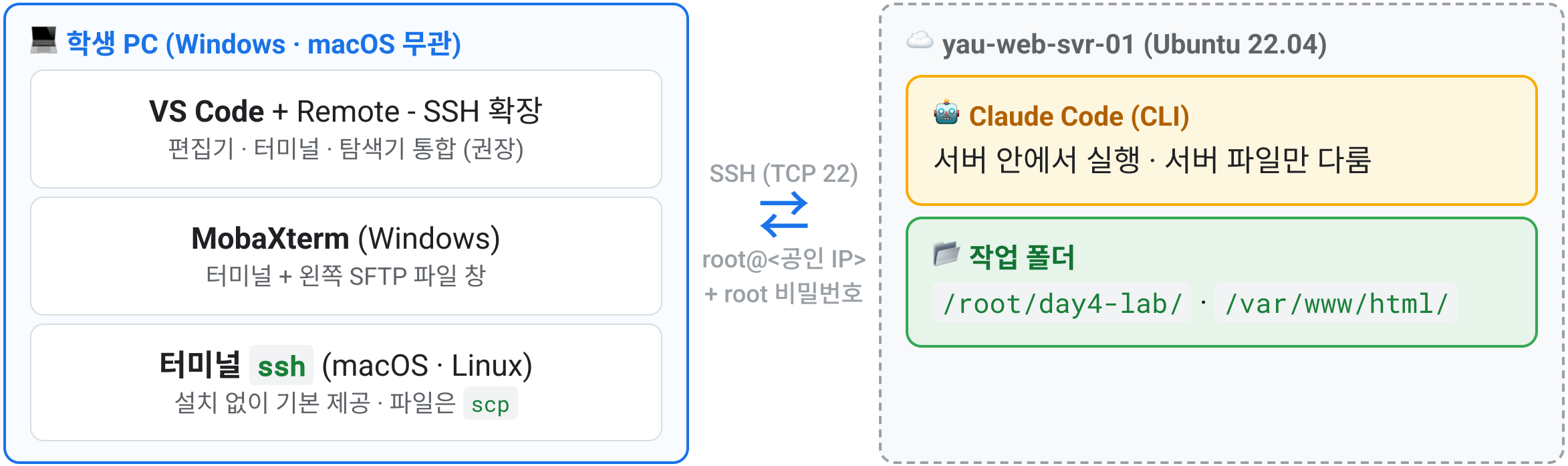
비교 항목	claude.ai (브라우저 · Day 2)	Claude Code (터미널 · Day 4)
실행 환경	웹 브라우저	서버 · PC의 **터미널**
작업 방식	코드를 **복사 → 붙여넣기** (사람이 직접)	파일을 **직접 만들고 · 고치고 · 실행**
파일 접근	파일을 **업로드**해야 읽음	작업 폴더의 파일을 **바로 읽음**
명령 실행	불가 (사람이 실행)	설치 · 점검 · 테스트 명령을 **직접 실행** (승인 후)
오류 처리	오류 메시지를 복사해 다시 질문	터미널 오류를 **스스로 읽고 수정**
기억 방법	대화창 · 프로젝트	`CLAUDE.md` (Part 5)

ℹ️ **Day 3 Cowork와 비교하면?** Cowork는 **내 PC 앱**에서 폴더를 맡기는 방식이고, Claude Code는 **터미널**에서 동작해 **화면 없는 서버**에서도 쓸 수 있습니다.

🔌 Part 3. 접속 구조

VS Code Remote SSH · MobaXterm · 터미널 ssh · 파일 주고받기

3.1 학생 PC와 서버의 연결 구조



- 학생 PC에는 **SSH 접속 도구만** 있으면 됩니다. Claude Code는 **서버에만** 설치합니다.
- 파일은 VS Code 탐색기 / MobaXterm SFTP 창에 **끌어다 놓기(Drag & Drop)**, Mac · Linux 터미널은 `scp` **명령**으로 주고받습니다.

3.2 접속 도구 비교

비교	VS Code + Remote - SSH	MobaXterm	터미널 `ssh`
지원 OS	Windows · macOS · Linux	Windows	macOS · Linux (기본 내장)
서버 파일 편집	VS Code 편집기로 바로 편집	내장 편집기 / 터미널	터미널 (`nano` 등)
터미널	VS Code 터미널 (`Ctrl + ` `)	기본 탭	터미널 앱 그 자체
접속 방법	`F1` → Connect to Host	Session → SSH	`ssh root@<공인 IP>`
파일 주고받기	탐색기 **Drag & Drop** · 우클릭 Download	SFTP 창 **Drag & Drop**	**`scp`** 명령
추천	✅ **기본 권장** (모든 학생 공통)	Windows 사용자 대안	**Mac · Linux 사용자 대안**

> 💡 **TIP**: 세 방법 모두 **Day 1의 실습 기록 카드**(공인 IP · root 비밀번호)만 있으면 접속할 수 있습니다.

🔑 Part 4. 설치 · 첫 실행 · 팀 계정 로그인

브라우저 없는 서버에서 로그인하는 법

4.1 Claude Code 설치 방법

방법	명령	특징
네이티브 설치 (권장)	`curl -fsSL https://claude.ai/install.sh | bash`	Node.js 불필요 · 한 줄 설치
npm 설치 (대안)	`npm install -g @anthropic-ai/claude-code`	Node.js 18 이상 필요

- 설치 확인: `claude --version` · 설치 진단: `claude doctor`
- 실행: 작업 폴더에서 `claude` 입력

> ⚠ **강의 전 확인**: 설치 명령 · 지원 OS · 요구 사항은 버전에 따라 바뀔 수 있습니다. **수업 전 Claude Code 공식 문서**에서 최신 설치 명령을 확인하고 슬라이드를 맞춰 주세요.

4.2 설치가 끝나면 이렇게 보입니다

```
❯ curl -fsSL https://claude.ai/install.sh | bash
Setting up Claude Code...

✔ Claude Code successfully installed!

  Version: 2.1.72

  Location: ~/.local/bin/claude


  Next: Run claude --help to get started
```

📋 설치 명령 (복사해서 붙여넣기)

```
curl -fsSL https://claude.ai/install.sh | bash
```

> 직접 타이핑하지 말고 **복사 → 붙여넣기** 하세요. 오타가 나면 설치가 실패합니다.

화면 문구	의미
`Setting up Claude Code...`	설치 진행 중
`✔ Claude Code successfully installed!`	**설치 성공**
`Version: 2.1.72`	설치된 버전 (화면은 예시 · 수업 당일 달라질 수 있음)
`Location: ~/.local/bin/claude`	실행 파일 위치 (root면 `/root/.local/bin/claude`)
`Next: Run claude --help`	다음 단계 안내

- 바로 `claude` 가 안 되면 → `source ~/.bashrc` 로 PATH를 다시 읽습니다.
- 확인: `claude --version` → 버전을 **기록 카드**에 적기

4.3 서버에서 팀 계정으로 로그인하는 흐름

① 서버 터미널
`claude` 실행
로그인 방식 선택
Claude 계정 (구독)

→

② 서버 터미널
인증 URL 표시
서버엔 브라우저가 없어 URL을 **복사**

→

③ 학생 PC 브라우저
URL 열고 승인
팀 계정으로 로그인 → 조직 선택 · 승인

→

④ 서버 터미널
인증 코드 붙여넣기
브라우저에 표시된 코드를 터미널에 붙여넣기

→

⑤ 확인
`/status`
계정 · 조직 · 모델 확인

- **팀(Team) 좌석**: 강사가 보낸 **초대 메일을 수업 전에 수락**해 두어야 합니다.
- API 키 방식은 사용하지 않습니다. (사용량 과금 → 팀 계정과 별도 비용)
- 로그인 정보는 서버의 `/root/.claude/` 에 저장 → **서버 이미지에도 함께 저장**됩니다.

4.4 첫 실행 4단계 — 처음 `claude` 를 입력하면

① 로그인 방식

구독 계정 선택

Claude account with subscription

(Pro · Max · Team)

→

② 브라우저 승인

URL → 승인

내 PC 브라우저에서 팀 계정 로그인 · **Authorize**

→

③ 보안 안내

주의사항 확인

"명령을 실행하고 파일을 수정할 수 있습니다" → Enter

→

④ 폴더 신뢰

Trust this folder

지금 폴더를 읽고 고쳐도 되는지 묻는 화면

- ①에서 **API 키(Console) 방식은 고르지 않습니다.** 사용량만큼 따로 과금됩니다.
- ②는 서버에 브라우저가 없으므로 **URL을 복사해 내 PC에서** 엽니다. (4.3)
- 화면 문구 · 순서는 버전에 따라 조금 다를 수 있습니다.

⚠ **④ 폴더 신뢰 = Day 3의 "전용 폴더"**

`/root` 나 `/` 에서 실행하지 말고 **`/root/day4-lab`** 에서 실행한 뒤 신뢰합니다. 신뢰한 폴더 안의 파일을 Claude가 읽고 · 고치고 · 실행합니다.

4.5 Claude Code 화면 구성 · 첫 대화 · 종료

⌨ 서버 터미널 — Claude Code 실행 화면 (구성 예시)

환영 상자
버전 · 모델 · 계정(조직) · 현재 폴더

대화 영역
내 요청 · Claude의 계획 · 실행한 명령 · 결과

입력창 `>`
자연어 요청 · `/` 명령 · `@파일`

상태 표시줄
권한 모드 (기본 · 수정 자동 허용 · 계획)

첫 대화 해 보기

```
안녕! 너는 이 서버에서 어떤 일을 할 수 있니?
지금 폴더에 어떤 파일이 있는지도 알려줘.
```

종료하는 방법

방법	입력
명령어 (권장)	`/exit`
키보드	`Ctrl + C` 두 번
키보드	`Ctrl + D`

💡 종료하면 대화 기억은 사라집니다. **다음에도 기억해야 할 것**은 `CLAUDE.md` 에 적습니다. → Part 5

4.6 꼭 알아둘 Claude Code 명령어

명령어	하는 일	명령어	하는 일
`/status`	계정 · 모델 · 버전 확인	`/init`	폴더 분석 → `CLAUDE.md` 생성
`/login` · `/logout`	로그인 · 로그아웃	`/memory`	`CLAUDE.md` 편집
`/help`	전체 명령어 보기	`/permissions`	허용 · 차단 규칙 관리
`/clear`	새 대화 시작	`/model`	모델 변경
`/compact`	긴 대화 요약	`/rewind`	이전 시점으로 되돌리기

키	하는 일
`Esc`	작업 중단
`Shift + Tab`	모드 전환 (기본 승인 → 수정 자동 허용 → 계획 모드)
`@파일명`	특정 파일을 대화에 첨부
`Ctrl + C` 두 번 / `/exit`	Claude Code 종료

> 💡 입력창에 `/` 만 입력하면 사용 가능한 명령 목록이 나타납니다. (버전에 따라 일부 다를 수 있음)

4.6 꼭 알아둘 Claude Code 명령어 — 슬래시(/) 명령

명령어	하는 일	명령어	하는 일
`/status`	계정 · 모델 · 버전 확인	`/init`	폴더 분석 → `CLAUDE.md` 생성
`/login` · `/logout`	로그인 · 로그아웃	`/memory`	`CLAUDE.md` 편집
`/help`	전체 명령어 보기	`/permissions`	허용 · 차단 규칙 관리
`/clear`	새 대화 시작	`/model`	모델 변경
`/compact`	긴 대화 요약	`/rewind`	이전 시점으로 되돌리기

4.6 꼭 알아둘 Claude Code 명령어 — 키보드

키	하는 일
`Esc`	작업 중단
`Shift + Tab`	모드 전환 (기본 승인 → 수정 자동 허용 → 계획 모드)
`@파일명`	특정 파일을 대화에 첨부
`Ctrl + C` 두 번 / `/exit`	Claude Code 종료

📘 Part 5. CLAUDE.md 이해하기

AI에게 주는 "업무 지침서" — 무엇이고, 어디에 두고, 어떻게 쓰나

5.1 CLAUDE.md란? — 세션이 끝나도 남는 프로젝트 기억

😵 문제: AI의 "기억 상실"

- `/exit` 로 종료하거나 새 터미널을 열면 **이전 대화 내용이 모두 사라집니다.**
- 새로 시작할 때마다 "이 서버는 무엇이고, 무엇은 하면 안 되고..."를 **매번 다시 설명**해야 합니다.

💡 해결: `CLAUDE.md`

- 작업 폴더에 두는 **마크다운(.md) 형식의 텍스트 파일**
- Claude Code가 **시작할 때마다 자동으로 먼저 읽는** 파일
- 그래서 **"AI에게 주는 업무 지침서", "프로젝트의 기억 노트"** 라고 부릅니다.

① `cd /root/day4-lab && claude`
새 세션 시작

▼ 자동

② `CLAUDE.md` **읽기**
규칙 · 금지 사항 · 작업 방식

▼

③ 규칙을 알고 시작
다시 설명하지 않아도 같은 기준으로 작업

ℹ **Day 3와 같은 아이디어**: Cowork의 **전역 지침 · 프로젝트 지침**이 Claude Code에서는 `CLAUDE.md` **파일**입니다.

5.2 무엇에 사용하나? — CLAUDE.md에 적는 4가지

① 프로젝트 설명

무엇을 하는 곳인가

"Day 4 실습 서버, Nginx로 `/var/www/html` 웹사이트 운영"

② 자주 쓰는 명령

확인 · 실행 방법

`nginx -t` · `systemctl reload nginx`
확인 주소 `http://<공인 IP>`

③ 규칙 · 금지 사항

지켜야 할 선

수정 전 백업 · `index_day1.html` 금지 · `rm -rf` 금지

④ 작업 방식

답변 스타일

한국어로 · 명령 전 한 줄 설명 · 끝나면 변경 파일 목록

CLAUDE.md가 있을 때	CLAUDE.md가 없을 때
세션마다 같은 규칙으로 시작	매번 규칙을 다시 설명
금지 사항을 미리 알고 피함	실수한 뒤에야 알려 줌
팀원 · 다른 서버와 같은 기준 공유 (Git)	사람마다 다른 결과

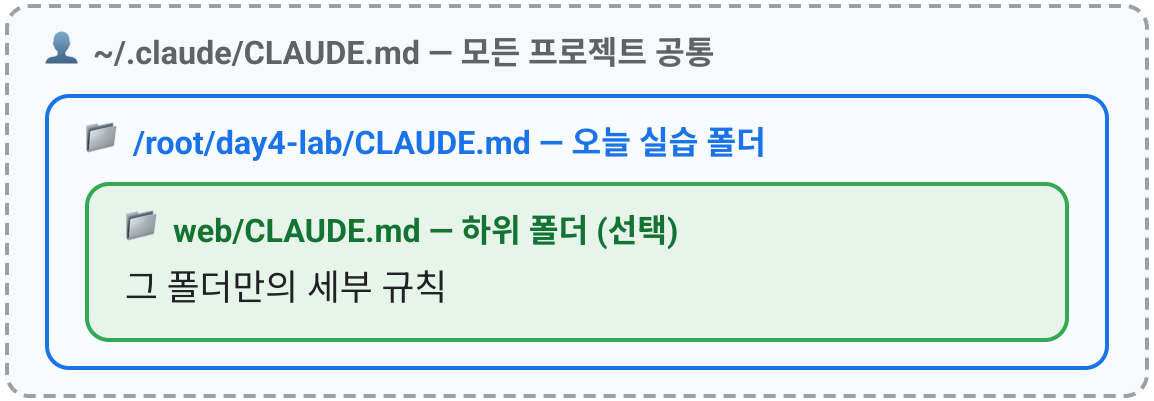
⚠ **넣지 말 것**: 비밀번호 · `.pem` 키 · API 키 같은 **비밀 정보** — CLAUDE.md는 Git에 올라가거나 다른 사람이 볼 수 있는 파일입니다.

5.2 무엇에 사용하나? — CLAUDE.md에 적는 4가지 — 있을 때 vs 없을 때

CLAUDE.md가 있을 때	CLAUDE.md가 없을 때
세션마다 같은 규칙으로 시작	매번 규칙을 다시 설명
금지 사항을 미리 알고 피함	실수한 뒤에야 알려 줌
팀원 · 다른 서버와 같은 기준 공유 (Git)	사람마다 다른 결과

5.3 어디에 두나? — 위치에 따라 적용 범위가 다르다

구분	위치	적용 범위	주로 적는 내용
사용자 전역	`~/.claude/CLAUDE.md`	이 계정의 **모든 프로젝트**	개인 취향 (한국어 답변 등)
프로젝트	`./CLAUDE.md` (작업 폴더)	**이 폴더 작업 시**	서버 · 프로젝트 규칙 ✅ 오늘
하위 폴더	`./web/CLAUDE.md` 등	그 폴더를 다룰 때 **추가로**	그 부분만의 세부 규칙



- 여러 개가 있으면 **함께 읽고**, 더 구체적인(안쪽) 규칙이 세부 사항을 보완합니다.
- 프로젝트 `CLAUDE.md`는 **Git에 함께 커밋** → 다른 사람 · 다른 서버도 같은 규칙 (Lab 06)

💡 **Day 3 대응표**: 전역 지침 ↔ `~/.claude/CLAUDE.md` · 프로젝트 지침 ↔ `./CLAUDE.md`

5.4 만드는 3가지 방법 — 직접 쓰기 · `/init` · `/memory`

방법 1 · 오늘 실습

직접 작성

터미널에서

```
cat << 'EOF'
```

또는 VS Code로 파일 편집

방법 2

`/init`

Claude가 폴더를 훑어보고
초안을 자동 생성

방법 3

`/memory`

Claude Code 안에서
CLAUDE.md를
열어 편집

`/init` 이 하는 일 (파일이 여러 개 있는 프로젝트에서 특히 유용)

1. 작업 폴더의 파일 · 폴더 구조를 **스스로 탐색**
2. 사용 언어 · 프로그램 · 실행 방법을 **파악**
3. 그 결과를 **`CLAUDE.md` 초안**으로 작성 → 사람이 **읽고 고쳐서** 완성

❗ **자동 생성 = 초안**: `/init` 이 만든 내용도 AI가 추측한 결과입니다. **금지 사항 · 안전 규칙은 사람이 직접** 추가합니다.

5.5 CLAUDE.md 기본 구조 — 오늘 서버용 예시

```
# Day 4 실습 서버 (yau-web-svr-01)

## 개요
- Ubuntu 22.04 + Nginx 웹서버
- 웹 파일: /var/www/html (index.html, calendar.html)

## 자주 쓰는 명령
- 설정 검사: nginx -t
- 적용: systemctl reload nginx
- 확인: curl -I http://localhost

## 작업 규칙
- 수정 전 같은 폴더에 백업본을 만든다
- 명령 실행 전 한국어로 한 줄 설명한다

## 금지
- index_day1.html 수정 · 삭제 금지
- rm -rf · SSH · 방화벽 · 계정 변경 금지
```

섹션	역할
개요	이 폴더 · 서버가 무엇인지
자주 쓰는 명령	확인 · 실행 방법을 정확히
작업 규칙	항상 이렇게 일해라
금지	절대 하지 마라

잘 쓰는 요령

- **짧고 구체적으로** — 한 줄에 한 규칙
- **명령형으로** — "~한다 / ~금지"
- 실수를 발견하면 **규칙을 한 줄 추가** → 같은 실수 반복 방지

5.5 CLAUDE.md 기본 구조 — 오늘 서버용 예시 — 예시 파일

```
# Day 4 실습 서버 (yau-web-svr-01)

## 개요
- Ubuntu 22.04 + Nginx 웹서버
- 웹 파일: /var/www/html (index.html, calendar.html)

## 자주 쓰는 명령
- 설정 검사: nginx -t
- 적용: systemctl reload nginx
- 확인: curl -I http://localhost

## 작업 규칙
- 수정 전 같은 폴더에 백업본을 만든다
- 명령 실행 전 한국어로 한 줄 설명한다

## 금지
- index_day1.html 수정 · 삭제 금지
- rm -rf · SSH · 방화벽 · 계정 변경 금지
```

5.6 CLAUDE.md의 한계 — 지침서이지 잠금장치가 아니다

✅ 할 수 있는 것
방향 잡기

작업 방식 · 금지 사항을 미리 알려
실수를 크게 줄임
세션 · 사람 · 서버가 바뀌어도
같은 기준

⚠️ 못 하는 것
100% 차단

AI가 규칙을 잘못 해석하거나
놓칠 수 있음
규칙이 길고 모호할수록 지키기
어려움

🛡️ 그래서
여러 겹의 안전장치

① CLAUDE.md 규칙
② 승인 화면 읽고 판단 (Part 6)
③ 수정 전 백업
④ 서버 이미지 (Part 7)

💡 **실습 연결**: Lab 04에서 `/root/day4-lab/CLAUDE.md` 를 만들고, "index_day1.html 을 지워줘"라고 요청해 Claude가 **규칙을 근거로 거절하는지** 확인합니다. `/memory` 로 적용 중인 파일도 열어 봅니다.

🛡️ Part 6. 권한과 안전 규칙

root 권한의 AI 에이전트를 어떻게 통제할까

6.1 root 권한 = 서버의 모든 것을 바꿀 수 있는 권한

- 학생은 서버에 **root**로 접속 → Claude Code도 **root 권한**으로 동작
- 가능한 일: 패키지 설치 · 서비스 재시작 · **모든 파일 수정 · 삭제**
- 그래서 Day 4 실습은 **학생 PC가 아닌 서버**에서 진행합니다.

> 🚫 **CAUTION**: 실습 서버에는 **개인 정보 · 비밀 키 · 중요한 원본**을 두지 않습니다.

1단계 · 승인
Claude Code가 실행 전 물어봄 → 사람이 확인

▼

2단계 · 규칙
`CLAUDE.md`에 금지 사항 명시

▼

3단계 · 백업
수정 전 파일 백업 (`cp`)

▼

4단계 · 서버 이미지
최후의 안전망: 서버 통째로 복구

6.2 권한 모드 3가지

모드	동작	언제 쓰나
기본 (승인 요청)	파일 수정 · 명령 실행 전 **매번 승인**을 요청	✅ **Day 4 기본값** — 무엇을 하는지 보며 배우기
수정 자동 허용	파일 편집은 자동 승인, 명령은 여전히 확인	익숙해진 뒤 반복 작업
계획 모드 (Plan)	실행하지 않고 **계획만** 세워 보여줌	큰 작업 전 방향 확인

- `Shift + Tab` 으로 모드를 바꾸고, 입력창 아래 표시로 현재 모드를 확인합니다.
- 승인 화면에서는 **Yes / 이번 세션 동안 허용 / No(다른 지시)** 중 선택합니다.

❗ **오늘의 규칙**: 승인 요청이 나오면 **명령을 읽고 이해한 뒤** 승인합니다. 모르는 명령은 "이 명령이 무엇을 하는지 설명해줘"라고 먼저 물어보세요.

6.3 해도 되는 일 · 승인이 필요한 일 · 금지

✅ 자유롭게
읽기 · 점검

파일 · 로그 읽기
디스크 · 메모리 · 서비스 상태 점검
`/root/day4-lab/` 안의 작업

⚠️ 확인 후 승인
변경 작업

`/var/www/html/index.html` 수정 (백업 후)
패키지 설치 (`apt-get install`)
Nginx 재시작 · 설정 변경

⛔ 금지
되돌리기 어려운 작업

`index_day1.html` 수정 · 삭제
`rm -rf` 등 대량 삭제
SSH · 방화벽 · 사용자 계정 변경

- 이 기준을 **`CLAUDE.md`** (프로젝트 규칙 파일)에 적어 두면 Claude Code가 작업할 때마다 참고합니다.
- `CLAUDE.md`는 Claude Code가 시작할 때 자동으로 읽는 **"AI에게 주는 업무 지침서"** 입니다.

6.4 CLAUDE.md 안전 규칙 예시 — Part 5를 오늘 서버에 적용

```
# Day 4 실습 서버 규칙 (yau-web-svr-01)

## 반드시 지킬 것
- 파일을 수정하기 전에 같은 폴더에 백업본을 만든다. (예: index_before_day4.html)
- 명령을 실행하기 전에 무엇을 하는 명령인지 한국어로 한 줄 설명한다.
- 작업이 끝나면 변경한 파일 목록과 확인 방법을 알려준다.

## 절대 하지 말 것
- /var/www/html/index_day1.html 수정 · 삭제 금지 (Day 1 백업본)
- rm -rf, 대량 삭제, 디스크 포맷 금지
- SSH 설정(/etc/ssh), 방화벽, 사용자 계정 · 비밀번호 변경 금지
```

💡 실습에서는 `/root/day4-lab/CLAUDE.md` 로 만들고, Claude Code는 **`/root/day4-lab` 폴더에서 실행**합니다.

💾 Part 7. 서버 이미지로 복구하기

연습장이 되려면 "되돌릴 수 있어야" 한다

7.1 서버 이미지 vs 스냅샷

비교	내 서버 이미지	스냅샷
단위	**서버 전체** (OS + 설치 프로그램 + 파일)	스토리지(디스크) 1개
복구 방법	이미지로 **새 서버 생성**	추가 스토리지로만 변환 가능
쓰임	✅ **망가진 서버를 통째로 되살리기**	부팅 불가 디스크에서 데이터 꺼내기
생성 시점	서버 **운영중**에도 생성 가능 (VPC)	스토리지 단위

> ℹ️ **VMware 스냅샷과 다른 점**
>
> VMware는 **같은 VM**을 그 시점으로 되돌리지만(Revert), Ncloud 서버 이미지는 이미지로 **새 서버를 만드는** 방식입니다. → 새 서버에는 **새 공인 IP**를 연결합니다.

7.2 망가진 서버 복구 절차

① 정지
망가진 서버
`yau-web-svr-01`
[서버 정지]

→

② 생성
이미지로 새 서버
최근 서버 이미지 선택
→
`yau-web-svr-02`

→

③ 연결
새 공인 IP
공인 IP 신청
→ 새 서버에 연결

→

④ 갱신
기록 카드
새 공인 IP 기록
SSH · 웹 접속 확인

→

⑤ 정리
옛 자원 반납
옛 공인 IP 반납
옛 서버 반납

- 이미지 안의 파일 · Nginx · **Claude Code 설치와 로그인 상태**가 새 서버에 그대로 들어갑니다.
- root 비밀번호는 [관리자 비밀번호 확인]으로 다시 확인합니다. (인증키 `yau-key-01.pem`)

🚫 **CAUTION**: ⑤를 빠뜨리면 **서버 · 공인 IP 요금이 두 배**로 나옵니다.

7.3 매일 수업 종료 30분 전 루틴

17:30 작업 저장 · Claude Code 종료 ▶ 서버 이미지 생성 ▶ 이미지 상태 **생성됨** 확인 ▶ 오래된 이미지 정리 ▶ 기록 카드 갱신

항목	규칙
이미지 이름	`yau-web-img-day4` (날짜 · Day 번호를 붙여 구분)
보관 개수	**최근 1~2개만** 보관, 오래된 이미지는 삭제 (보관 용량만큼 비용 발생)
복구 기준점	가장 최근 이미지 = **전날(또는 오늘) 수업 종료 시점**
중간에 망가지면	최근 이미지로 복구 → 필요 시 Claude Code만 다시 설치 · 로그인

> 💡 매일 같은 시간에 같은 절차를 반복하며 **클라우드 운영의 "백업 루틴"** 을 몸으로 익힙니다.

🐙 Part 8. GitHub의 역할 & 실습 미리보기

서버는 이미지로, 작업물은 GitHub로

8.1 서버 이미지 vs GitHub — 역할이 다르다

비교	서버 이미지	GitHub
무엇을 남기나	**서버 전체 상태**	**내가 만든 작업물(코드 · 문서)**
목적	망가진 서버 **되살리기**	작업물 **제출 · 공유 · 포트폴리오**
위치	네이버클라우드 계정 안	인터넷 (서버 밖)
서버를 반납하면	이미지 삭제 시 함께 사라짐	**계속 남음**
오늘	필수	선택

🚫 **GitHub에 절대 올리지 말 것**: `.pem` 인증키 · root 비밀번호 · API 키 · `/root/.claude/` 폴더

8.2 Day 4 실습 진행 순서

Lab	내용	결과물	구분
00	서버 상태 확인 · 기록 카드 준비	운영중 서버	필수
01	VS Code Remote SSH / MobaXterm / 터미널 접속 · 파일 주고받기	`/root/day4-lab/`	필수
02	Claude Code 설치	`claude --version`	필수
03	팀 계정 로그인	`/status` 확인	필수
04	안전 규칙 (`CLAUDE.md`) · 권한 모드	규칙 파일	필수
05	Claude Code로 **하루 할 일 메모 앱** (v1~v4 필수 · v5~v6 과제)	📝 `memo.html` + 서버 저장	필수
06	GitHub Push	내 저장소	과제
07	종료 30분 전: 서버 이미지 생성 · 정리	`yau-web-img-day4`	필수
08	서버 복구 절차	새 서버	참고

8.3 실습 미리보기 — Claude Code로 만드는 하루 할 일 메모 앱

v1
메모 화면
`memo.html`
브라우저에 저장

→

v2
달력 연동
Day 3 달력
날짜 → 그날 메모

→

v3
서버 API
Flask
`memos.json`

→

v4
Nginx 연결
PC · 폰 어디서나
같은 메모

→

v5~v6 · 과제
서비스 · 완성도
systemd · 개수 배지
백업 · 비밀번호

질문	답
왜 단계로 나누나?	작게 만들고 → 확인하고 → 다음 단계 (Day 2 바이브 코딩의 리듬)
"서버에 데이터를 남긴다"는?	메모가 **내 브라우저**가 아니라 **서버 파일**(`/var/lib/memo-app/memos.json`)에 저장
무엇이 안전장치인가?	`CLAUDE.md` 메모 앱 규칙 · 수정 전 백업 · `nginx -t` · 승인 읽기 · 서버 이미지

✅ 셀프 체크 퀴즈

1. Claude Code를 학생 PC가 아니라 **서버**에 설치하는 이유 2가지는?
2. 브라우저가 없는 서버에서 팀 계정으로 로그인할 때, 인증 URL은 **어디서** 여나요?
3. Claude Code가 명령 실행 승인을 요청할 때, 모르는 명령이라면 어떻게 하나요?
4. 망가진 서버를 **통째로** 되살리는 도구는 서버 이미지와 스냅샷 중 무엇인가요?
5. 서버 이미지로 복구한 뒤 **반드시 반납**해야 하는 자원 2가지는?
6. `CLAUDE.md` 는 **언제** 읽히고, 주로 **무엇**을 적나요?

> **정답**: ① 환경 통일 · 학생 PC 보호 (+ Day 1~3 서버 연속성) ② 학생 PC 브라우저 ③ "이 명령이 무엇을 하는지 설명해줘"라고 먼저 묻고 이해한 뒤 승인 ④ 서버 이미지 ⑤ 옛 공인 IP · 옛 서버 ⑥ Claude Code가 시작할 때 자동으로 읽음 · 프로젝트 설명 · 자주 쓰는 명령 · 작업 규칙 · 금지 사항

Q & A

수고하셨습니다! 잠시 휴식 후 핸즈온 실습을 시작합니다.

강사: 신인철

소속: 클라우드스퀘어